

Ratelplus Build Plan

Sep 28, 2026 · @Muhammad

This is the developer documentation for Ratelplus's own network and business software: what each component does, how they connect, and the order we build them in. The target is the end-to-end demo on **Nov 26, 2026**: a line sold in RatelDesk browses on LTE, makes VoLTE calls, is charged for its data and is topped up on RatelPay, all on real SIMs. RatelCore's data path already works in the lab and the radio joins it this week, so every developer can start now.

Start with **Ratel Plus Subscriber Platform PRD**. It says what we are building and why, in product terms. This document is the how. Where the two disagree about what the product does, the PRD wins.

The components

Every part has a name, and the code, the tickets and this document all use it.

Name	What it is	Built on	Runs on	Owner
RatelCore	LTE core: MME, HSS, SMF, PCRF, SGW-U and UPF	Open5GS v2.8.0	core-cp, core-up	Network team
RatelVoice	VoLTE core: P-CSCF, I-CSCF, S-CSCF, media relay and IMS DNS	Kamailio IMS, rtpengine	voice (new VM)	Network team
RatelOps	Monitoring, alarms and the live dashboard	Prometheus, Grafana, Alertmanager	ops (new VM)	Network team
RatelLink	Provisioning API; the only writer to RatelCore's subscriber database	Python, FastAPI, MongoDB driver	core-cp	Software team
RatelMeter	Data usage and call records for billing	Python	Agents on core-up and voice; API on bss-app	Software team

Name	What it is	Built on	Runs on	Owner
RatelBSS	Business engine: customers, stock, plans, lines, charging, wallets, vouchers, payments, reports	Python, FastAPI, PostgreSQL, Redis	bss-app	Software team
RatelDesk	Staff web app	React, TypeScript	bss-app	Software team
RatelPay	Customer top-up page	A small page served by RatelBSS, light enough for 3G	bss-app, public through a reverse proxy	Software team

Why we are building this

Ratelplus is a licensed operator with its own Band 40 spectrum and three LTE radios covering one site. Until now the core network and billing software behind those radios came from an outside vendor, licensed by subscriber count. The cap we were given was far below the capacity we understood we had agreed, each subscriber above it carried a fee, and in September we lost admin access altogether.

The lesson is simple. Whoever controls the core controls the network. So we are building our own on open-source software we control: Open5GS for RatelCore, which already runs in the lab, and Kamailio for RatelVoice. The radios stay, because they work with any standards-compliant core.

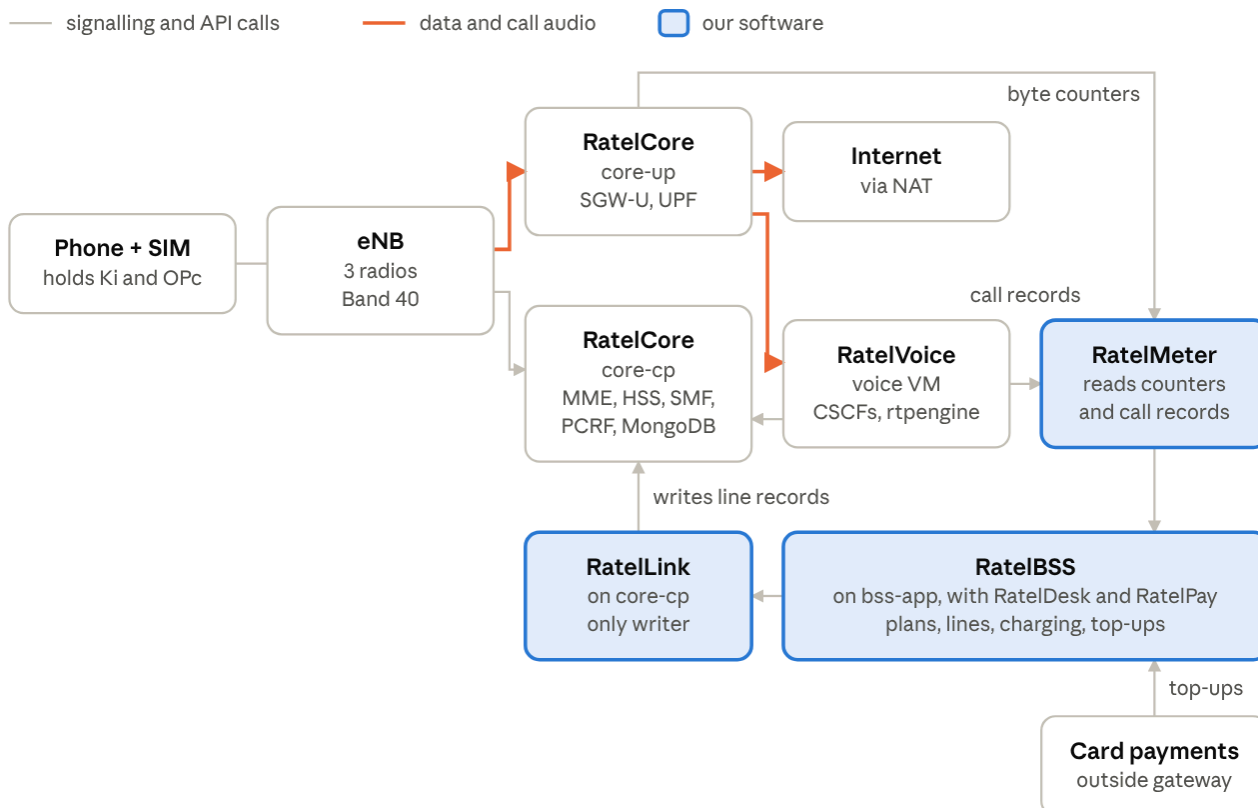
Four principles follow from that, and they explain most of the decisions in this document:

- **Nothing we build can be switched off from outside.** No licence files, no subscriber caps, no vendor remote access.
- **Data and voice both ship for the demo, on separate tracks.** The software team builds the business side and the network team builds RatelVoice. The two meet at one point: RatelLink writes a line's calling settings when it activates the line.
- **SIM keys and money get the most care.** A leaked key compromises a line for the life of its SIM, and a billing error loses customers' trust faster than an outage does.
- **The simplest thing that works on 26 November, with a clear path to more.** No Kubernetes, no message bus, no microservices beyond what the network forces.

How the network works

You don't need to be a telecom engineer to build the business side, but you do need this picture in your head. Five machines run everything, and the business software touches the network at two points only: RatelLink and RatelMeter.

Our software touches the network at exactly two points



the network and our software · two touch points

Signalling decides whether a phone may use the network; traffic is the phone's actual data and call audio. RatelLink is the only thing that ever writes to RatelCore's subscriber database, and RatelMeter only reads byte counters and call records. Everything else in our software talks to those two services, never to the network.

Machine	Address	Runs	Our service on it
core-cp	192.168.1.184	RatelCore control plane: MME, HSS, SMF, PCRF, MongoDB	RatelLink
core-up	192.168.1.162	RatelCore user plane: SGW-U, UPF	RatelMeter's agent, data mode
voice	Not yet created	RatelVoice: Kamailio P-CSCF, I-CSCF and S-CSCF, rtpengine, IMS DNS, MySQL	RatelMeter's agent, call mode

Machine	Address	Runs	Our service on it
bss-app	Not yet created	PostgreSQL, Redis	RatelBSS, RatelMeter's API, RatelDesk, RatelPay
ops	Not yet created	RatelOps: Prometheus, Grafana, Alertmanager	None

What happens when a phone attaches

1. The phone finds a cell broadcasting our network, PLMN 621-00, and sends an attach request carrying its IMSI.
2. The radio (eNB) forwards the request to the MME on core-cp.
3. The MME asks the HSS to authenticate the SIM. The HSS reads the line's record from MongoDB and uses the SIM's secret keys, Ki and OPc, to build a challenge. The keys themselves never leave the core.
4. The phone answers using the same keys stored on its SIM. The right answer proves the SIM is genuine.
5. The core opens an internet session. It gives the phone an IP address, applies the speed limits (AMBR) from the line's record, and tells the UPF on core-up to forward the phone's packets.
6. From then on the phone's data flows from the eNB through the UPF to the internet, and the UPF counts every byte.

What happens when a phone makes a call

1. After attaching, a VoLTE phone opens a second session on the ims APN. The SMF gives it the P-CSCF's address.
2. The phone registers with RatelVoice: P-CSCF, then I-CSCF, then S-CSCF. The S-CSCF checks the SIM with the same HSS on core-cp, over the Cx interface. The line's calling identity comes from its IMSI and the MSISDN in its record.
3. When the phone dials, the S-CSCF finds the called line's registration and routes the call to it. The called phone shows the caller's MSISDN.
4. The P-CSCF asks the PCRF, over the Rx interface, for a dedicated voice bearer (QCI 1), so call audio gets priority over internet traffic.
5. The audio flows through rtpengine on the voice VM. When the call ends, the S-CSCF writes a call record, which RatelMeter picks up.

Why changes wait for the next attach

Steps 3 and 5 of the attach are the only moments RatelCore reads a line's record. Change the record while a session is running and nothing happens until the phone attaches again, which can be days later. That is why suspending a line or changing its speed only takes effect at the next attach, until RatelLink's live changes land. Calls follow the same rule: a phone stays registered for calls until it registers again or reattaches.

Start here

The rule for this week: nothing you build may depend on the radio. The eNB joins RatelCore on Friday. Until then, build and test against the lab core, which already carries a working data path end to end: core-cp at 192.168.1.184 runs the control plane (MME, HSS, SMF, PCRF) and core-up at 192.168.1.162 runs the user plane (SGW-U, UPF).

That works because the business software touches the network at exactly two points. RatelLink is how anything tells the network that a line exists, whether it may make calls, and how fast its internet may go. RatelMeter is how the network tells the business what each line used. Customers, plans, vouchers, payments and reports all live purely in software and never touch the network directly.

Write those two seams down as an OpenAPI spec before anyone writes application code. Once they exist, the RatelBSS developers code against a mock of RatelLink, RatelMeter emits records in the agreed shape, and nobody waits on anybody.

Voice starts this week too, on its own track. RatelVoice is network work: the network team builds it, and it touches our software only through RatelLink, which writes each line's calling settings, and RatelMeter, which reads call records.

The two contracts to freeze on day one

Both contracts are plain REST: standard HTTP status codes and JSON bodies, and on an error a body of `{"error": {"code": "...", "message": "..."} }`. Every call carries an API key in the Authorization header, one key per calling system. Every call that changes something accepts an Idempotency-Key header.

Contract 1: RatelLink

Method	Path	Body	Effect on the network
POST	/v1/sims	imsi, ki, opc	Stores the SIM's keys in RatelLink. No service yet

Method	Path	Body	Effect on the network
POST	/v1/lines/{imsi}/activate	msisdn, speed_ul_mbps, speed_dl_mbps, voice	Writes the subscriber document: keys, MSISDN, the internet APN at the given speeds, and the ims APN when voice is true
POST	/v1/lines/{imsi}/data	mode (full, slow or off), speed_ul_mbps, speed_dl_mbps	Changes the internet APN only: its speed for full or slow, removed for off. Calling is untouched
POST	/v1/lines/{imsi}/deactivate	reason	Removes service: no internet and no calls
GET	/v1/lines/{imsi}		Status, APNs, IP address, speeds and MSISDN. Never keys
GET	/v1/assignments		Every current IP address with its IMSI, for RatelMeter

Three decisions are baked into this contract:

- **Ki and OPc live only on the network side.** The SIM import is split: keys go to RatelLink, and RatelBSS receives only IMSI and ICCID. RatelBSS's database never holds a key, so the most sensitive data sits in the smallest, most locked-down service.
- **Running out of data never touches calling.** A line that runs out has its internet slowed or removed through /data and keeps its ims APN, so calls still work. Only /deactivate stops calls.
- **Deactivation is delete and rewrite, for now.** RatelLink keeps keys in its own store, deletes the Open5GS subscriber document on deactivate, and rewrites it on activate. Open5GS v2.8.0 also carries a subscriber_status barring field. Test whether the MME rejects a barred subscriber, and switch to it if it does, since it's cleaner.

Until RatelLink's live changes land, every change here takes effect at the line's next attach, not mid-session. Nothing in RatelBSS may assume otherwise.

Contract 2: RatelMeter

RatelBSS reads two feeds from RatelMeter:

Method	Path	Query	Returns
GET	/v1/usage	from, to, imsi (optional)	Data records: imsi, period_start, period_end, bytes_up, bytes_down
GET	/v1/calls	from, to, msisdn (optional)	Call records: call_id, from_msisdn, to_msisdn, started_at, answered_at, ended_at, end_reason

```
{
  "records": [
    {"imsi": "6210000000000001", "period_start": 1790424000, "period_end": 1790424300,
      "bytes_up": 5242880, "bytes_down": 47185920}
  ],
  "next_cursor": null
}
```

Times are Unix epoch seconds in UTC, sent as integers, and usage is in bytes. A call that was never answered has answered_at set to null. Both feeds return at most 5,000 records per response, with next_cursor for the rest.

Behind /v1/usage, store a richer internal record: record_id, imsi, ue_ip, apn, period_start, period_end, bytes_up, bytes_down, source. Write one record per line per 300-second interval. Key every record on imsi plus period_start, so a collector restart that replays an interval overwrites instead of double counting, and key calls on call_id for the same reason. That single rule is most of the difference between billing you can defend and billing you can't.

Workstreams

Seven workstreams, each named after what it builds. RatelLink and RatelMeter are the network side of the software and implement the two contracts. RatelBSS splits in two and never touches the network except through those contracts. RatelDesk and RatelPay sit on top of RatelBSS. RatelVoice and RatelOps belong to the network team.

Workstream	Builds	Talks to	Start now?
RatelLink	Contract 1. Owns the key store and every write to RatelCore's subscriber database, including each line's calling settings. Later, live changes to connected lines	MongoDB on core-cp	Yes
RatelMeter	Contract 2. An agent that reads per-line byte counters on core-up and call records on voice, the record store, and the two read endpoints	core-up, voice, RatelLink's /v1/assignments	Yes for data, against the lab data path. Calls once RatelVoice writes records
RatelBSS: lines	Staff accounts, customers, SIM and number stock, plans, bundles, lines and their states	RatelLink, through Contract 1	Yes, against a mock of RatelLink
RatelBSS: money	The rating job that turns usage into allowance deductions, wallets, vouchers, one payment gateway, reports	RatelBSS: lines, and RatelMeter through Contract 2	Vouchers and the gateway sandbox now, rating once the plan model settles
RatelDesk and RatelPay	Every screen for the two RatelBSS workstreams, plus the public top-up page	RatelBSS's API	Yes, against a mock of the API
RatelVoice	Kamailio IMS, rtpengine, IMS DNS, the ims APN and Rx on RatelCore, and phone testing	RatelCore's HSS, SMF and PCRF	Yes, this week

Workstream	Builds	Talks to	Start now?
RatelOps	Prometheus, Grafana, Alertmanager, alarms and the demo dashboard	Every machine, read only	From week four

RatelBSS: money is where the business actually happens. It reads what RatelMeter measured, decides against the plan the line is on, and, through the line state machine, has RatelLink change the line's service when it hits zero. Until live changes land, that change only bites at the next attach. That is acceptable for the demo, and customers must be told honestly at launch.

Splitting the team

- **One backend developer:** RatelLink first, then a thin slice of RatelBSS: lines (customer, SIM, one plan, activate), then RatelMeter, then RatelBSS: money. Live changes wait.
- **Two:** one owns the network side of the software (RatelLink, RatelMeter, later live changes), because it's Linux, MongoDB and packet counters. The other owns RatelBSS, which is pure application code. They write the two contracts together on day one.
- **Three:** RatelLink plus RatelMeter, then RatelBSS: lines, then RatelBSS: money.
- **Four:** one per workstream, from RatelLink to RatelBSS: money.

RatelDesk and RatelPay need a frontend developer from week two. With fewer than four backend developers, whoever owns RatelBSS: lines builds its screens as well.

RatelVoice needs one engineer full time from this week, working with the network lead. It is not a side task for a backend developer: it is SIP, Diameter and Linux networking, and most of its risk sits in testing real phones.

Whichever split you choose, the owner of RatelLink should be your most careful developer, not your fastest. That service holds every line's keys.

Component specifications

Each spec answers the same questions in the same order: why the component exists, what it owns, the data it keeps, the rules it must never break, and the test that proves it is done. The contracts above are the interface; these specs describe what sits behind them. Where a spec and a contract disagree, the contract wins and the spec gets fixed.

RatelLink

Why it exists. RatelCore trusts whatever is in MongoDB. Making one small service the only writer gives us three things: SIM keys live in one locked-down place, RatelBSS never needs to know Open5GS's internal schema, and if we ever change core software, only this service changes.

What it owns. The SIM key store, and every write to the subscribers collection in the open5gs database, including the ims APN and MSISDN that RatelVoice depends on. It runs on core-cp and reaches MongoDB on localhost only.

Data it keeps. In a separate database, ratel_link, in the same MongoDB, with its own user. The Open5GS database is touched only for subscriber documents.

Collection	Key fields	Notes
sim_key	imsi (unique), ki, opc, amf, created_at	ki and opc encrypted at rest, with the key held outside the database
line_state	imsi, status, msisdn, apns, ue_ip, speed_ul_mbps, speed_dl_mbps, data_mode, updated_at	Our record of what we last wrote to Open5GS
audit_log	at, api_key_id, action, imsi, before, after	Append-only, and never contains ki or opc

status is one of provisioned (keys stored, no service), active (subscriber document written) or barred. Whether barring is a flag or a deleted document depends on the subscriber_status test in week one. data_mode is full, slow or off.

Interfaces. The Contract 1 endpoints. GET /v1/lines/{imsi} returns status, APNs, IP address, speeds and MSISDN, never keys, so RatelBSS can check its view against the network's. GET /v1/assignments lists every current IP address with its IMSI; RatelMeter's agent caches it and refreshes it every interval.

Rules.

- Ki and OPc are write-only. No endpoint returns them, no log line contains them, and they never reach RatelBSS.
- Every call is idempotent. Activating an active line with the same settings succeeds and changes nothing.
- Build each Open5GS document from a template taken from a subscriber Open5GS created itself, one that already has the ims APN the network team set up. Never hand-write the schema.

- Give every line a fixed IPv4 address on the internet APN at activation, from the 10.45.0.0/16 pool the lab already routes and NATs. No line is activated without one, and a released address isn't reused for 24 hours, so a late counter reading can never be charged to the wrong person. The ims APN uses its own pool, which the network team sets.
- Every change goes into the audit log with the calling system's key id.

Done when.

- A line created through the API attaches through our radio, browses, and registers for calls.
- Its Open5GS document matches the template field for field, apart from its own values.
- A test searches every log file after a full test run and finds no Ki or OPc value.

Live changes. Without them, a line that runs out of data or is suspended keeps browsing and calling until its phone reattaches, which can be days later. That is revenue lost, and it undermines every plan we sell. It is P1 in the PRD (CHG-6), so the spike comes first and the build follows only if the spike shows it is small. Answer three questions against Open5GS v2.8.0, in writing, in week two:

1. When RatelLink changes a subscriber document, does the HSS tell the MME, with Insert Subscriber Data or Cancel Location? If it does, much of this already exists.
2. Can the PCRF push a new policy, such as a lower speed, to a live session over Gx? If not, is a small patch or an Rx client the simpler route?
3. Can we force a detach for a single IMSI, so the phone reattaches and picks up its new record? A detach also ends its registration for calls.

The build throttles through the PCRF and cuts through a forced detach, both triggered inside RatelLink so the contract stays unchanged. Done when a line moved to out_of_data mid-session is slowed within 60 seconds, and a suspended line loses internet and calls within 60 seconds. If the spike shows either target can't be met in time, the fallback of changes at next attach is written down, and customer care knows it before launch.

RatelMeter

Why it exists. Open5GS produces no usage records, and without them we cannot bill. RatelMeter turns the UPF's byte counters, and RatelVoice's call records, into the records RatelBSS reads.

What it owns. Two parts. One agent program runs in data mode on core-up, reading the counters, and in call mode on the voice VM, reading call records. The RatelMeter API runs on bss-app, stores the records and serves /v1/usage and /v1/calls.

Data it keeps.

Store	Key fields	Notes
usage_record in PostgreSQL	imsi plus period_start (unique), period_end, ue_ip, apn, bytes_up, bytes_down, source, record_id	One row per line per 300-second interval
call_record in PostgreSQL	call_id (unique), from_msisdn, to_msisdn, started_at, answered_at, ended_at, end_reason	One row per call
Last reading, on core-up	ue_ip, bytes_up_total, bytes_down_total, read_at	Used to work out each interval's increase
Spool, on core-up and voice	One file per interval or batch of calls	Records the agent could not deliver, replayed in order

How data works. Every 300 seconds the agent reads the per-address counters on the ogstun interface and subtracts the previous reading. It maps each address to its IMSI using the list RatelLink publishes at GET /v1/assignments, then posts the batch to the RatelMeter API. Packets from the phone's address count as upload, and packets to it count as download.

How calls work. The S-CSCF writes a record when each call starts and ends, using Kamailio's accounting (the acc and dialog modules), into RatelVoice's MySQL. Every 60 seconds the agent reads new records, joins each call's start and end into one record, and posts it to the RatelMeter API. The network team owns the Kamailio side; agree the record's fields with them in week two.

Rules.

- Align intervals to the clock (00:00, 00:05, 00:10 and so on) in UTC, so a replayed interval always lands on the same key.
- Never lose an interval or a call. If the RatelMeter API is unreachable, spool to disk and replay in order.
- Never double count. The API upserts on imsi plus period_start for data and on call_id for calls, so a replay overwrites instead of adding.
- Handle counter resets. A reading lower than the last one means the UPF or its host restarted, so count the new reading from zero rather than recording a negative increase.
- Skip intervals with zero bytes, which keeps the table small.
- Keep the clocks on core-up and voice synchronised over NTP, because every record's key and every call's duration depend on them.

Done when.

- Over a real session, the records' totals match the interface's own counters within 1%.
- Every test call appears once in /v1/calls, with its duration correct to the second.
- Stopping the agent mid-interval and restarting it loses nothing and counts nothing twice.
- Taking the RatelMeter API offline for an hour and bringing it back delivers every spooled interval and call.

RatelBSS: lines

Why it exists. This is the business itself: who each customer is, which SIM and number they hold, which plan they bought and what state their line is in. Customer care and sales spend their day in it, through RatelDesk.

What it owns. PostgreSQL on bss-app, holding staff accounts, customers, SIM and number stock, plans, bundles and lines. Its API serves RatelDesk and RatelPay under /v1/customers, /v1/lines, /v1/plans, /v1/bundles and /v1/stock. It never talks to the network; every network change goes through RatelLink.

Entities.

Entity	Key fields	States
customer	customer_id, name, phone, email, id_type, id_number, address, kyc_status, blocked	kyc_status is pending, verified or rejected; blocked is a separate flag
sim	iccid, imsi, batch	in_stock, in_use, retired
number	msisdn	available, in_use, quarantined
plan	plan_id, name, price_kobo, validity_days, data_bytes, speed_ul_mbps, speed_dl_mbps, channels, on_exhaustion, slow_ul_mbps, slow_dl_mbps	on_sale, retired
bundle	bundle_id, name, price_kobo, validity_days, data_bytes, channels	on_sale, retired
line	line_id, customer_id, iccid, imsi, msisdn, plan_id, expires_at, state_before_suspension	See below

on_exhaustion is either slow or stop. A number taken back from a terminated line sits in quarantined for whatever period compliance sets before it can be sold again.

Line states. Every change calls RatelLink with what the network should now do:

From	To	When	Call to RatelLink
awaiting_activation	active	First plan paid for and KYC verified	/activate with the plan's speeds and voice true
active	out_of_data	Allowance used up before the plan expires (RatelBSS: money decides)	/data with mode slow and the plan's slow speeds, or mode off, per on_exhaustion
out_of_data	active	Customer buys a bundle or a new plan	/data with mode full and the plan's speeds
active or out_of_data	expired	Plan validity ends without renewal	/deactivate
expired	active	Customer buys a plan	/activate
active or out_of_data	suspended	Customer care action, with a reason	/deactivate
suspended	the state it had before	Customer care action, with a reason	/activate, then /data if it was out_of_data
Any other state	terminated	Care action or end of contract	/deactivate; SIM retired, number quarantined

The out_of_data state lets customer care see at a glance why a customer is slow or offline even though their plan is still valid, and it is the state in which internet stops but calls carry on.

Rules.

- RatelBSS never holds Ki or OPc. A SIM import is split: the keys go to RatelLink, and RatelBSS keeps ICCID, IMSI and batch.
- State changes happen only through the state machine, and each one writes an audit entry saying who made it and why. No code updates a state column directly.
- No line activates until KYC is verified. Capture the customer's NIN at registration. The exact verification rules are compliance's call, so confirm the current NCC SIM registration requirements with them before the pilot.
- Money is stored as whole kobo, never as a decimal.

Done when.

- The sell-to-activate flow runs against RatelLink and produces exactly one /activate call.
- Tests prove every transition not in the table above is refused.
- Customer care can find any line by MSISDN, ICCID or IMSI in under a second.

RatelBSS: money

Why it exists. It turns usage into money. It knows each line's allowance and balance, deducts what the line uses, takes top-ups, and decides when the line has run out.

What it owns. Allowance buckets, wallets, the ledger, vouchers and payments, all in PostgreSQL on `bss-app`, plus the rating job.

Data it keeps.

Table	Key fields	Notes
allowance_bucket	bucket_id, line_id, source (plan or bundle), data_bytes_total, data_bytes_used, valid_from, valid_to	One per plan period or bundle bought
wallet	line_id, balance_kobo	Cash the customer can spend on plans and bundles
ledger_entry	entry_id, line_id, amount_kobo, kind, reference, created_at	Append-only. A wallet's balance always equals the sum of its entries
voucher	card_number, pin_hash, face_value_kobo, batch_id, state, expires_at, redeemed_by, redeemed_at	The PIN is stored hashed, never in plain text
payment	payment_id, provider, provider_ref (unique), amount_kobo, status, line_id	One row per provider transaction

The rating loop. Every 300 seconds, the loop takes the usage records it has not yet rated and deducts each one from the line's buckets, soonest expiry first. It marks each record rated in the same database transaction as the deduction, so a crash can never rate a record twice. When the last bucket runs out, it moves the line to `out_of_data` through the line state machine, which calls RatelLink. When a plan's validity ends without renewal, it moves the line to `expired` the same way. Data left in an expired bucket is lost; there is no

rollover in the first release. Calls are not rated for the demo: their records show on the line page, and charging for minutes (VOI-7) is designed after the demo, because it needs a check when a call starts.

Because the loop runs every five minutes, a line can use up to about five minutes of data past zero before it is noticed. Until live changes land, the cut then waits for the next attach. Both are acceptable for the demo and are covered under what not to build yet.

Vouchers. A 20 digit card number, a 13 digit PIN plus one Luhn check digit, and the states requested, approved, sent_to_printer, in_stock, on_sale, used and expired, with locked for lost or stolen cards. Allow at most 5 failed redemption attempts per number per hour.

Payments. One gateway for the first release. Credit a payment only after verifying the webhook's signature and re-querying the transaction with the provider. Because `provider_ref` is unique, a webhook delivered twice credits once.

Tax. Apply VAT at the rate finance confirms (7.5% at the time of writing, but check), and record it as its own ledger entry so reports can separate it.

Done when.

- The full prepaid loop runs in the lab: buy a plan, use it, run out, slow down or lose internet at the next attach while calls still work, top up, get full speed back.
- A webhook delivered twice and a voucher redeemed twice each credit exactly once.
- A nightly job confirms every wallet balance equals the sum of its ledger entries, and raises an alert on any mismatch.

RatelDesk and RatelPay

Why they exist. RatelDesk is where staff do everything. RatelPay is the only part of the platform customers see.

What they own. The screens and nothing else. Every rule lives in RatelBSS, and the screens call its API.

Rules.

- RatelDesk is served only on the office network and the VPN.
- RatelPay shows nothing personal about a line's owner (TOP-2) and loads in under 3 seconds on 3G (TOP-3), so keep it a small page without a heavy framework bundle.
- Every screen that changes data has a history tab (ACC-3).

Done when. The four journeys in the PRD run through the screens without anyone touching the API by hand.

RatelVoice

Why it exists. It gives every Ratelplus line calls from the phone's normal dialler, over LTE. Without it, a line is data only.

What it is. Three Kamailio IMS roles (P-CSCF, I-CSCF and S-CSCF), rtpengine to carry each call's audio, a DNS server for the IMS domain, and MySQL for Kamailio's own tables. It all runs on a new VM, voice, on the same LAN as core-cp with no NAT between them, because the P-CSCF and the PCRF must see each other's real addresses.

Where to start. The open-source [docker_open5gs](#) project runs exactly this pairing of Open5GS and Kamailio. Take its Kamailio, rtpengine and DNS configuration as the reference, not its Open5GS, since ours already runs. The [Open5GS VoLTE tutorial](#) covers the same setup step by step.

Changes on RatelCore.

- Use RatelCore's own HSS as the IMS HSS, over Cx. Open5GS's HSS supports this ([reported working with Kamailio](#)), and it keeps each line in one record that RatelLink writes once. It builds the line's IMS identities from its IMSI and MSISDN. PyHSS is the fallback if Cx gives trouble.
- Add the ims APN to the subscriber template: QCI 5 for signalling, with its own IP pool on the SMF.
- Give the SMF the P-CSCF's address, so phones learn it when they open the ims session.
- Peer the PCRF with the P-CSCF over Rx, so each call gets a dedicated QCI 1 bearer.
- The IMS domain is ims.mnc000.mcc621.3gppnetwork.org. A two-digit MNC is written with three digits in IMS names.

The real risk is the phones. Many Android phones switch VoLTE on only for networks their maker has listed, and some makers limit it to networks they recognise. Ours is new, so some phones will refuse. Some phones also expect voice settings on the SIM, and many expect IPsec between the phone and the P-CSCF, which Kamailio's P-CSCF supports. Find out which phones work in the first two weeks, before anything is built on top of them.

Rules.

- Calls only for lines RatelLink has activated with voice on. RatelVoice keeps no line records of its own beyond Kamailio's registrations.
- Every call leaves a record for RatelMeter.
- The configuration lives in the repository under network/voice, so every change is reviewed and can be rolled back.

Done when. VOI-1 to VOI-4 in the PRD pass: at least two phone models we would sell register and show VoLTE, 20 test calls of 3 minutes each all connect with two-way audio and none drops, the called phone shows the caller's number, and a deactivated line can't call or be called after its next attach.

RatelOps

Why it exists. It tells the network team something is wrong before customers do, and it is the live screen at the demo.

What it is. Prometheus, Grafana and Alertmanager on a small ops VM. Open5GS's MME and SMF already export [Prometheus metrics](#), including attached phones. Kamailio has a Prometheus module for registrations and calls, node_exporter covers each machine, and a radio counts as up when the MME sees it connected. Alerts go to the network team by email and one messaging channel the team picks.

Done when. OPS-1 and OPS-2 in the PRD pass.

The eight-week plan

Each week ends with a gate: a test that either passes or doesn't. A week isn't finished until its gate passes, and the next week's work assumes it has. Three tracks run side by side: RatelCore and the radio, RatelVoice, and the software.

Week	Dates	RatelCore and radio	RatelVoice	Software	Gate
1	28 Sep to 2 Oct	Turn on MongoDB authentication; create the bss-app and voice VMs; cable the radio and bring it up on RatelCore	Kamailio P-CSCF, I-CSCF and S-CSCF, rtpengine and IMS DNS installed; ims APN and P-CSCF address on RatelCore; test phones collected	Contracts merged on day one, then the week one tasks below	The eNB registers with our MME, and openapi.yaml is merged
2	5 to 9 Oct	Settle SIM keys; first real-SIM attach	First phone registers for calls on a real SIM; phone test list started; call record fields agreed with RatelMeter	RatelLink complete against the lab, writing calling settings; RatelMeter's data agent running on core-up; RatelBSS's line flow calling the real RatelLink; live-changes spike written up	A real SIM, created through RatelLink, attaches, browses and registers for calls

Week	Dates	RatelCore and radio	RatelVoice	Software	Gate
3	12 to 16 Oct	Production security settings on the core, including ciphering; NTP on every host	First call between two Ratelplus phones, with the QCI 1 voice bearer set up through Rx. Voice decision point on 16 October	RatelMeter writes records every 300 seconds and serves /v1/usage; plans and bundles; rating logic tested on synthetic records	Usage for a real session matches the interface counters within 1%, and two phones hold a call
4	19 to 23 Oct	Create the ops VM; Prometheus and Grafana on RatelCore	Call records flowing to RatelMeter's call agent; Kamailio metrics in Prometheus	Rating job live, with out_of_data triggered at zero; vouchers redeem; first RatelDesk screens	The full prepaid loop works in the lab, and a line sold in RatelDesk makes a call
5	26 to 30 Oct	Alertmanager alarms; the RatelOps demo dashboard; any core changes the spike calls for	Phone test list final; IPsec and codec fixes for the phones that need them	Live changes, if the spike says they are small; the payment gateway end to end in sandbox; first reports; RatelPay	Alarms fire when a radio, RatelCore or RatelVoice is switched off
6	2 to 6 Nov	Backups of MongoDB, PostgreSQL and MySQL, and a real restore rehearsal	20 calls at once; restart tests	Hardening: restart, replay and idempotency tests; security review of RatelLink and RatelBSS: money	Restarts and replays lose nothing and count nothing twice; a restore has actually been done
7	9 to 13 Nov	Pilot on staff SIMs; capacity tests for attach bursts	Pilot: staff use their lines for calls; the 100-call test	Daily fixes from the pilot; reconciliation reports	A full week of pilot usage matches the network's counters, and 98 of 100 test calls connect
8	16 to 20 Nov	Demo runbook; plan for moving existing customers	Fixes from the pilot	Customer care trained on RatelDesk; final fixes; two full demo rehearsals	The demo script runs twice with no step repeated
Demo	23 to 26 Nov	Freeze	Freeze	Freeze, on call	The demo on 26 November

Three things outside the software team can move the whole plan. **SIM keys gate week two:** until the core holds Ki and OPc for real SIMs, no real phone can attach, and the same keys decide whether existing customers can move across later. **Phones gate voice:** if by 16 October no phone we could sell registers and calls reliably, management decides whether

the demo shows calls on the phones that do work, or whether voice moves to a later date. Data does not wait for voice. **The network team's load:** it now carries RatelCore, RatelVoice, RatelOps and the radio, so RatelVoice needs its own engineer this week.

Week one in detail

Day one, everyone. Write openapi.yaml holding both contracts, review it together, merge it, and stand up a mock server from it so the RatelBSS developers can start the same afternoon. No application code until that file is merged.

RatelLink

- ☐ Take the subscriber document shape from a record Open5GS created itself and use it as the template. When the network team adds the ims APN, take the template again from a subscriber that has it. Do not hand write the schema
- ☐ A key store with Ki and OPc encrypted at rest, and the key held outside the database
- ☐ /v1/sims, /activate, /data and /deactivate against the lab MongoDB
- ☐ Test whether v2.8.0 enforces subscriber_status barring, and write the answer down
- ☐ Start the live-changes spike from RatelLink's spec. No production code; the write-up is due at the end of week two

Done when a subscriber written through the API matches the template field for field, apart from its own values. The first real-SIM attach, this Friday if SIM keys are ready and next week if not, must use a line created through this API, not by hand. That attach is RatelLink's acceptance test.

RatelMeter

- ☐ Confirm on day one, with RatelLink's owner, the fixed-address rule in RatelLink's spec: every line gets a static IP from the pool, written by RatelLink at activation. That turns the IMSI to IP join into a lookup. The cost is sizing the pool for every line rather than those attached at once. The fallback is parsing the SMF's session log, which works but breaks quietly whenever the log format moves
- ☐ Spike the counter mechanism on core-up's ogstun interface: nftables per-element counters, per-line accounting rules, or conntrack accounting. Pick the one that loses no bytes under thousands of short flows, and measure it rather than assuming
- ☐ The data agent writing 300-second records keyed on imsi plus period_start, with /v1/usage on top

Done when the agent's totals for a session match the interface's own byte counters within 1%. If the old emulator VM still exists, test with it. If not, build against recorded counter snapshots and run the accuracy check on Friday's first real traffic. The call mode starts in

week two, once the network team has agreed the call record's fields.

RatelBSS: lines

- ☐ Postgres schema for customer, sim, number, plan, bundle and line, carrying the state machine in the spec above, including suspended
- ☐ The line flow: create a customer, assign a free SIM and number, pick a plan, activate. Activation calls RatelLink's mock with the plan's speeds and voice true
- ☐ /v1/customers and /v1/lines endpoints

Done when that whole flow runs against the mock and produces exactly the /activate call the contract defines.

RatelBSS: money

- ☐ Voucher batches: 20 digit card number, 13 digit PIN plus a check digit, the full state machine, and redemption with a per-number attempt limit
- ☐ One payment gateway in sandbox. Verify the webhook signature, then re-query the transaction with the provider before crediting anything
- ☐ Rating logic as a pure function over usage records and a plan allowance, tested with synthetic records

Done when a voucher and a sandbox payment each credit a balance exactly once, even when the gateway delivers the same webhook twice.

RatelVoice

- ☐ Create the voice VM on the same LAN as core-cp, with no NAT between them
- ☐ Kamailio P-CSCF, I-CSCF and S-CSCF, rtpengine and IMS DNS, starting from the docker_open5gs reference configuration
- ☐ On RatelCore: the ims APN (QCI 5) with its own pool, the P-CSCF address on the SMF, Rx between the PCRF and the P-CSCF, and Cx between the HSS and the I-CSCF and S-CSCF
- ☐ Collect at least six test phone models, including the brands our customers carry, and ask the SIM supplier whether our SIMs support VoLTE

Done when the three Kamailio roles run and their Diameter connections to the HSS and the PCRF are up. The first phone registration needs the radio, so it lands in week two.

Stack and repo layout

Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2 with Alembic migrations, PostgreSQL, and Redis for idempotency keys, rate limits and the background job queue. RatelLink uses the MongoDB driver directly, since that's where Open5GS keeps subscribers. RatelDesk is React and TypeScript. No Kafka, no Kubernetes, no microservices beyond what the network forces: three deployables, run with Docker Compose or systemd. RatelVoice and RatelOps are configuration, not application code, and live in the same repository.

Deployable	Contains	Runs on	Talks to
ratel-link	RatelLink	core-cp	MongoDB on 127.0.0.1 only
meter-agent	RatelMeter's agent: data mode on core-up, call mode on voice	core-up and voice	Pushes records to the app; spools to local disk when the app is unreachable, so nothing is ever lost
app	RatelBSS, RatelMeter's API, and the RatelDesk and RatelPay front ends, as separate modules in one process	bss-app	PostgreSQL and Redis on the same VM, RatelLink over the LAN

Two placement rules matter more than they look. First, the business software and the voice core each get their own VM. Create bss-app, voice and ops on the DL380 rather than putting them on the core hosts, because a bad deploy must never be able to take RatelCore down with it. Second, only RatelLink ever touches MongoDB, and it does so over localhost. MongoDB holds every line's Ki and OPc and runs without authentication by default, so enable authentication, keep it bound to 127.0.0.1, and update db_uri in hss.yaml and pcrf.yaml to match. That's a network task, not a developer one, but nothing goes near real SIMs until it's done.

One repository, because the contract is shared and a change to it has to land on both sides in the same commit:

```

ratel/
  contracts/openapi.yaml      the two contracts, the source of truth
  services/
    ratel_link/              deploys to core-cp
    meter_agent/             data mode on core-up, call mode on voice
    app/
      bss_lines/             RatelBSS: lines
      bss_money/             RatelBSS: money
      meter_api/             serves /v1/usage and /v1/calls
  web/
    desk/                    RatelDesk
    pay/                      RatelPay
  network/
    voice/                   RatelVoice configuration
    ops/                     RatelOps configuration
  deploy/
    core-cp/  core-up/  voice/  bss-app/  ops/

```

CI runs lint, unit tests, and a contract test that exercises each running service against `openapi.yaml` (Schemathesis does this directly from the spec), so drift between the contract and the code fails the build instead of reaching Friday. API keys and the credential store's encryption key live in environment files with owner-only permissions on each host, never in the repository.

Engineering rules

These apply to every workstream. Each one is cheap to follow from the first commit and expensive to retrofit after launch.

Area	Rule
SIM keys	Ki and OPc exist only in RatelLink's store and in Open5GS's MongoDB. Never in PostgreSQL, logs, tickets, chat, test fixtures, or backups that leave core-cp unencrypted
Access	Every API and RatelDesk are reachable only over the WireGuard VPN. Two parts are public by design: RatelPay and the payment gateway's webhook. Expose only those paths, verify every webhook signature, and rate-limit both. One API key per calling system, rotated every 90 days

Area	Rule
Call records	Who called whom and when is personal data. Same access rules as customer records, and never exported without a logged reason
Money	Whole kobo, never floating point. The ledger is append-only: a correction is a new entry, never an edit
Idempotency	Every endpoint that changes state or moves money accepts an Idempotency-Key header. Repeating a call with the same key returns the first result
Time	Store and send UTC, as epoch seconds on the wire. Show West Africa Time only in RatelDesk and RatelPay
Logs	Structured JSON. Never log keys or full PINs; mask card and PIN numbers to their last four digits
Testing	Rating, state machines and check digits get unit tests. Contract tests run against openapi.yaml in CI. Anything that touches the network is tested against the lab core, never production
Review	Changes to RatelLink (keys) and RatelBSS: money need two reviewers. Everything else, including RatelVoice configuration, needs one
Environments	The lab is today's core-cp and core-up. Decide before week 7 whether the pilot runs on these machines or new ones. Either way, no developer has write access to production databases

Definition of done

A task is done when all of these are true:

- It is merged through review with CI passing.
- If it changes an API, `openapi.yaml` changed in the same commit.
- If it touches the network, it has been shown working against the lab core, not only in unit tests.
- If it runs in production, it has a short runbook note covering how to tell it is healthy and what to do when it isn't.

What not to build yet

Each of these is real and each will be asked for. None belongs before the demo.

- **Calls to other networks, SMS and emergency calls.** Interconnect and a message centre depend on agreements that take months, and emergency calls wait on compliance. All three come before commercial voice, not before the demo.
- **Charging for calls.** Calls between Ratelplus lines are free for the demo. Minutes need a check when a call starts, which is designed after.
- **Live-change code before the spike answers its questions.** The right design depends on what v2.8.0 actually does, and guessing means writing it twice.
- **Dealer hierarchy, dealer wallets and balance transfer.** Sell through Ratelplus's own channels first.
- **USSD.** The highest-value channel for prepaid, but it runs through a short code and an aggregator agreement that take weeks to arrange. Start that paperwork now and build after the demo.
- **A second payment gateway.** One is enough to prove the flow.
- **Postpaid invoicing and credit limits.** Prepaid first. It's simpler and it's where the customers are.
- **Real-time online charging for data.** The 300-second rating loop is enough for a first release. Build online charging only when the loop's delay starts costing money you can measure.
- **Self-care and dealer apps.** RatelDesk and RatelPay cover the demo, as the PRD describes.

The finish line

The demo is done when a customer can be sold a SIM in RatelDesk, browse through RatelCore, call another Ratelplus line through RatelVoice, run out of data and be slowed or cut off while calls still work, and top up on RatelPay, with every byte and every call matching the network's own records. Everything in this plan serves that sentence, and anything that doesn't can wait.

Glossary

The telecom terms you'll meet in this document and in the code, and where each one shows up.

Term	Meaning	Where you'll meet it
AMBR	Aggregate Maximum Bit Rate: the speed cap on a line's traffic, one for upload and one for download	speed_ul_mbps and speed_dl_mbps in Contract 1; ambr in the Open5GS document
APN	Access Point Name: the network a session connects to. internet for data, ims for calls	The subscriber document and RatelLink
Attach	The moment a phone registers with the network and opens a session	The only time RatelCore reads a line's record
CDR	Call Detail Record. For data, one line's usage over one 300-second interval; for voice, one call	Contract 2 and RatelMeter
CSCF	Call Session Control Function. The P-CSCF is the phone's first contact, the I-CSCF finds the right S-CSCF, and the S-CSCF registers phones and routes calls	RatelVoice
Cx	The interface the I-CSCF and S-CSCF use to check a line with the HSS	Between voice and core-cp
eNB	The radio base station. We have three at the site	The network diagram
HSS	Home Subscriber Server: checks that a SIM is genuine, using its keys, for both data and calls	core-cp, reading MongoDB
ICCID	The serial number printed on the SIM card	SIM stock in RatelBSS
IMS	IP Multimedia Subsystem: the standard for carrying calls over mobile data. RatelVoice is our IMS	RatelVoice
IMSI	The SIM's 15 digit identity on the network. Ours start with 62100	The key for everything in RatelLink and RatelMeter
Ki, OPc	The two secret keys held on the SIM and in the core, which together prove a SIM is genuine	RatelLink's key store only

Term	Meaning	Where you'll meet it
MME	Mobility Management Entity: handles attach and signalling	core-cp
MSISDN	The line's phone number, and its identity when calling	Number stock in RatelBSS; the subscriber document
NIN	National Identification Number, captured for SIM registration	KYC in RatelBSS
PCRF	Policy and Charging Rules Function: applies speed and priority to live sessions, including the voice bearer	RatelLink's live changes; Rx for RatelVoice
PLMN	The network's identity: country code plus network code. Ours is 621-00	Broadcast by the eNB; set in the core's configuration
QCI	QoS Class Identifier: a session's traffic priority. Internet uses 9, call signalling 5 and call audio 1	The subscriber document; RatelVoice
rtengine	Relays the audio of each call	The voice VM
Rx	The interface the P-CSCF uses to ask the PCRF for a voice bearer	Between voice and core-cp
Session	The connection a phone holds while attached, with its IP address and speed limits. Also called a bearer	What live changes change
SGW-U, UPF	The user plane: forwards the phone's packets to and from the internet, and counts the bytes	core-up and RatelMeter
SMF	Session Management Function: opens sessions, assigns IP addresses and tells phones where the P-CSCF is	core-cp
TAC	Tracking Area Code: the area a cell belongs to. Ours is 1	Core configuration
UE	User Equipment: the phone or router holding the SIM	Everywhere
VoLTE	Voice over LTE: phone calls carried as data over our LTE network, through RatelVoice	Everywhere voice comes up